

PLC Implementation in GX Works2

Adaptive Demand Allocation Architecture (ADA) — Programming Reference

Ladder Logic, Tag Nomenclature, and an Experimental Structured Text (ST) Translation

Author Sambridda Ranabhat
Organisation Mechatronics Engineering Pvt. Ltd. (MEPL)
Location Kathmandu, Nepal
Date July 17, 2026
Status Programming Reference — For Review

This document is a programming reference companion to the *Thermofluid Analysis of Architecture II* and its PLC Implementation section: nomenclature, a consolidated tag/ID lookup, every ladder logic block in one place, and an experimental Structured Text translation flagged accordingly. Design rationale, derivations, and open items live in the parent architecture document, not here.

1 Nomenclature

1.1 Instruction Set (FX-Series-Style, via GX Works2)

Table 1: Ladder instruction abbreviations used throughout this document.

Instruction	Meaning
LD / LDI	Load contact (normally-open / normally-closed) – starts a rung
AND / ANI	Series AND (normally-open / normally-closed)
OR / ORI	Parallel OR (normally-open / normally-closed)
OUT	Drive a coil/flag
SET / RST	Latch ON / latch OFF (retains state until explicitly reset)
MOV	Move/copy a value into a register
MUL / DIV / ADD / SUB	16-bit arithmetic
DMUL / DADD / DDIV	32-bit (double-word) arithmetic – required when a value exceeds the 16-bit range
INC	Increment a register by 1
CMP	Compare, sets a 3-bit result (>, =, <)
PLS	One-scan pulse on a rising edge
FROM / TO	Read from / write to an intelligent module's buffer memory
K	Prefix for a constant (decimal)

1.2 Device Prefixes

Table 2: PLC device types.

Prefix	Device type
X	Physical digital input
Y	Physical digital output
M	Internal relay/flag (not a physical point)
D	Data register (16-bit unless part of a 32-bit pair)
T	Timer
AI-#	Analog input channel (placeholder label, pending real tag numbers)

1.3 Process and Architecture Terms

Table 3: ADA-specific abbreviations.

Term	Meaning
ADA	Adaptive Demand Allocation Architecture
P1 / P2 / P3	Priority tiers: T ₅ trim-heat / PO tank maintenance / T ₁₀ palm oil melt
LSM / HSM / FSM	Trunk flow states: Low/High/Full Supply Mode – 1, 2, or 3 destinations served simultaneously
PO tank	Palm olein tank – whichever of TA/TB/TC is currently designated
S_{casual} , $S_{\text{immediate}}$	Tank scoring functions (volume-weighted / temperature-weighted)
SV	Solenoid valve (water routing)
PBV	Pneumatic ball valve (oil dispense)
HE1–3	Auxiliary immersion heating elements, 12 kW each
AODD	Air-operated double-diaphragm pump (T ₁₀ → T ₅ transfer)
VFD-01 / VFD-02	Trunk circulation drive / oil distribution drive
T_f	Dynamic upper thermal boundary (T ₅ safety cutoff)
Eco/Efficiency/Standard/Full Throttle	P3 operating-posture modes, driven by T ₅ 's level (α_S)
P_{base} , E_{req}	10-second rolling average heating power and the throttled power request derived from it (Dynamic Power Evaluation, Section 3.11)

2 Tag ID Tables

2.1 Digital I/O (Confirmed)

Table 4: Digital inputs.

Tag	Correspondence	Notes
X0	E-STOP	NC, hardwired safety assumed upstream
X1	Automation Enable/Disable	
X2	Circulation Pump (manual)	
X3	Oil Distribution Pump (manual)	
X4	Jacket Pump (manual)	Unused – Y4 runs continuously, see Section 2.1 no
X5	AODD Pump (manual)	
X6	Float – HEA, NC	Warning only
X7	Float – SOL, NC	Warning only
X10	Loop Exit	Manual-mode only
X_Confirm	PO tank designation confirm	Momentary, HMI or push-button
X_EStopReset	E-STOP reset	Momentary, must follow physical E-STOP clear
X_AlarmReset	Master alarm acknowledge	Momentary, HMI or push-button

X_Confirm, X_EStopReset, and X_AlarmReset are used by name in the ladder logic (Sections 3.6, 3.3, 3.23) but, like the AI channels, have not been assigned real numeric addresses – add them here once confirmed, keeping the naming consistent with X0–X10 above.

Table 5: Digital outputs.

Tag	Correspondence	Notes
Y0	E-STOPPED lamp	
Y1	Automation Enabled lamp	
Y2	Circulation Pump – HSM	
Y3	Oil Distribution Pump active	
Y4	Jacket Pump active	Always on with M2_SystemOK – no manual/auto split
Y5	AODD Pump active	
Y6	Circulation Pump – LSM	
Y7	Heating Element 1	12 kW
Y10	Heating Element 2	12 kW
Y11–Y14	SV100–SV103	T5/TA/TB/TC branch valves
Y15/Y16	SV104/SV105	Solar selector: through/bypass
Y17	Circulation Pump – FSM	
Y20	Heating Element 3	12 kW
Y21–Y24	PBV100–PBV103	T5/TA/TB/TC oil valves
Y25	Solar Cycle Active lamp	

Y4 (Jacket Pump active) intentionally does not follow the Manual/Automatic Gating Pattern (Section 3.4) used by the other pumps. By design it runs continuously whenever the system is not E-STOPped, with no manual/auto split – see the Jacket Pump subsection immediately after the gating pattern (Section 3.5). X4 (Jacket Pump manual) is therefore intentionally unused; it is not read anywhere in the program.

2.2 Analog Inputs (Channel Count Confirmed, Tags Pending)

Table 6: Analog inputs.

Channel	Correspondence	Register
AI-1–4	Level: T5/TA/TB/TC (4-20mA ultrasonic)	$\alpha_S, \alpha_{10A,B,C}$
AI-5–8	Temp: T5/TA/TB/TC (4-20mA Pt100)	$\theta_S, \theta_{10A,B,C}$
AI-9/10	Temp: Solar loop / Thermal battery	$\theta_{\text{Sol}}, \theta_H$
AI-11	Flow: Trunk/outlet (4-20mA, confirmed)	Flow rate

2.3 Data Registers (D) by Function

Table 7: Register groups – see the full architecture document for exact per-register purpose within each group.

Group	Range/registers
AI scaling	D0, D2, D4 (per-channel, repeated pattern)
PO designation	D50 (pending), D51 (active)
T_{water}	D200–D204, D_Ptotal, D_Twater_target
Mode	D_Mode, D_AlphaS
Suspicion	D_Suspicion
Dynamic power	D_Etotal, D_Pbase, D_Twater, D_TempDiff, D_TempCalc – feeds D_Ereq_kW below (Section 3.11)
Aux staging	D_AuxStage, D_Ereq_kW
Scoring	D410–D418, D_ScoreCasualXX, D_ScoreImmXX, D_EligCasualXX, D_MaxScore, D_MaxID, D_RankedTankTemp
Mass conversion	D510, D520, D_MassT5/TA/TB/TC, D_HeightT5/TA/TB/TC
T_f	D330–D371, D_ThetaH, D_ThetaS, D_Mp, D_Tf
Arbiter	D_NeedCount, D_ServeCount
Fault detection	D600–D622, D_DeltaThetaXX, D_MaxHeatRateXX, D_MaxCoolRateXX, D_DeltaMassXX

D_Twater is a separate register from **D_Twater_target**: the latter is the 10s-refreshed setpoint computed in Section 3.8; the former is the measured water-loop temperature consumed by Dynamic Power Evaluation (Section 3.11). Neither the AI table above nor the Tf block's θ_H/θ_S registers currently account for where D_Twater is measured or scaled from – confirm its source (a dedicated AI channel, or a copy of an existing one) before wiring this block.

2.4 Working Flags (M) by Function

Table 8: Flag groups.

Group	Flags
Safety/mode	M1_ESTOP, M2_SystemOK, M3_AutoEnabled
PO designation	M0–M3
Demand	M_P1Demand, M_P2Demand, M_D_T5/TA/TB/TC
Arbiter	M_Serve_T5/TA/TB/TC, M_Challenger
T_f /P1	M_TfInterlock, M4_ThrottleOrDivert, M_BatchRelease
P2	M_Overrun_Pulse, M_PromptActive, M_HMIPrompt, M_Alarm, M_DisableFeed
Oil distribution	M_DemandX/Y/Z, M_ServeX/Y/Z
Fault detection	M_FaultHeatingXX, M_FaultCoolingXX, M_FaultLevelXX, M_HighLimitXX, M_TAActiveTransfer

2.5 Timers (T)

Table 9: Timers.

Timer	Purpose
T0	10s refresh oscillator (self-resetting)
T5	Suspicion meter 5-min no-response auto-increment

Every address above is an illustrative placeholder, coordinated only to avoid collisions with itself – confirm a real, complete address map before programming.

3 Ladder Logic – Full Listing

3.1 First-Scan Initialization

```
// M8002 is the Mitsubishi First-Scan normally open contact
LD    M8002

// 1. Clear Master Alarms and Latches on Power-Up
RST   M_Alarm
RST   M_HighLimitTA
RST   M_HighLimitTB
RST   M_HighLimitTC

// 2. Clear Fault Flags
RST   M_FaultHeatingTA
RST   M_FaultCoolingTA
RST   M_FaultLevelTA
RST   M_FaultHeatingTB
RST   M_FaultCoolingTB
RST   M_FaultLevelTB
RST   M_FaultHeatingTC
RST   M_FaultCoolingTC
RST   M_FaultLevelTC

// 3. Reset Critical Counters and State Registers
MOV   K0 D_Suspicion
MOV   K0 D_AuxStage
MOV   K0 D_Ereq_kW
MOV   K0 D_NeedCount
MOV   K0 D_ServeCount
```

3.2 AI Channel Read/Scale

```
FROM  K0 K10 D0 K1
MUL   D0 K100 D2
DIV   D2 K4000 D4
```

3.3 Master Safety and Mode Control

```
LDI   X0
SET   M1_ESTOP
LD    X_EStopReset
RST   M1_ESTOP
LD    M1_ESTOP
OUT   Y0
LDI   M1_ESTOP
OUT   M2_SystemOK

LD    X1
```

```
OUT Y1
LD X1
OUT M3_AutoEnabled
```

3.4 Manual/Automatic Gating Pattern (repeat per pump/valve)

```
LD M2_SystemOK
ANI M3_AutoEnabled
AND X2
OUT [pump output]

LD M2_SystemOK
AND M3_AutoEnabled
AND [arbiter / need-array condition]
OUT [pump output]
```

This pattern is instantiated explicitly below for the trunk circulation pump (Y2/Y6/Y17, via the Need-Array Arbiter's serve count) and for the oil distribution pump (Y3, via the Oil Distribution block). The jacket pump (Y4) deliberately does *not* use this pattern – see the following subsection.

3.5 Jacket Pump

Simplified by design: the jacket pump runs continuously whenever the system is not E-STOPped, with no manual/auto split and no dependency on X4.

```
LD M2_SystemOK
OUT Y4
```

3.6 PO Tank Designation

```
LD X_Confirm
PLS M100
LD M100
MOV D50 D51

LD= D51 K0 OUT M0
LD= D51 K1 OUT M1
LD= D51 K2 OUT M2
LD= D51 K3 OUT M3
```

3.7 10-Second Refresh Oscillator

```
LDI T0
OUT T0 K100
LD T0
OUT M10_Refresh
```


3.8 T_{water} Target

```
LD M10_Refresh
MUL D_Ptotal K300 D200
DIV D200 K6160 D202
ADD D202 K400 D204

LD< D204 K400
MOV K400 D204
LD> D204 K700
MOV K700 D204
MOV D204 D_Twater_target
```

3.9 Mode Register

```
LD= D_Mode K0 AND< D_AlphaS K85 MOV K1 D_Mode
LD= D_Mode K1 AND>= D_AlphaS K90 MOV K0 D_Mode
LD= D_Mode K1 AND< D_AlphaS K65 MOV K2 D_Mode
LD= D_Mode K2 AND>= D_AlphaS K70 MOV K1 D_Mode
LD= D_Mode K2 AND< D_AlphaS K45 MOV K3 D_Mode
LD= D_Mode K3 AND>= D_AlphaS K50 MOV K2 D_Mode
```

3.10 Suspicion Score

```
LD M_Overrun_Pulse
INC D_Suspicion
LD> D_Suspicion K3
MOV K3 D_Suspicion

LD= D_Suspicion K1 OUT M_HMIPrompt
LD= D_Suspicion K2 OUT M_Alarm
LD= D_Suspicion K3 OUT M_DisableFeed
LD= D_Suspicion K3 OUT M_Alarm

LD M_PromptActive
OUT T5 K3000
LD T5
INC D_Suspicion
RST M_PromptActive

LD M_DailyReset_Pulse
MOV K0 D_Suspicion
```

3.11 Dynamic Power Evaluation

```
// Calculate Base Power over 10s Window
LD M10_Refresh
DDIV D_Etotal K10 D_Pbase

// Condition 1: T < 40.0 C (Full Power)
```

```

LD M10_Refresh
AND< D_Twater K400
MOV D_Pbase D_Ereq_kW

// Condition 2: T >= 70.0 C (Zero Power)
LD M10_Refresh
AND>= D_Twater K700
MOV K0 D_Ereq_kW

// Condition 3: 40.0 C <= T < 70.0 C (Proportional Throttling)
LD M10_Refresh
AND>= D_Twater K400
AND< D_Twater K700
SUB K700 D_Twater D_TempDiff
MUL D_Pbase D_TempDiff D_TempCalc
DIV D_TempCalc K300 D_Ereq_kW

```

3.12 Auxiliary Stage Register

```

LD= D_AuxStage K0 AND> D_Ereq_kW K0 MOV K1 D_AuxStage
LD= D_AuxStage K1 AND<= D_Ereq_kW K2 MOV K0 D_AuxStage
LD= D_AuxStage K1 AND>= D_Ereq_kW K14 MOV K2 D_AuxStage
LD= D_AuxStage K2 AND< D_Ereq_kW K10 MOV K1 D_AuxStage
LD= D_AuxStage K2 AND>= D_Ereq_kW K26 MOV K3 D_AuxStage
LD= D_AuxStage K3 AND< D_Ereq_kW K22 MOV K2 D_AuxStage

LD>= D_AuxStage K1 OUT Y7
LD>= D_AuxStage K2 OUT Y10
LD>= D_AuxStage K3 OUT Y20

```

3.13 Tank Scoring (worked for TA; repeat for TB, TC)

```

MUL D_ThetaTA K100 D410
DIV D410 K700 D_TnormTA

MUL D_AlphaTA K6 D412
MUL D_TnormTA K4 D414
ADD D412 D414 D_ScoreCasualTA

MUL D_AlphaTA K4 D416
MUL D_TnormTA K6 D418
ADD D416 D418 D_ScoreImmTA

```

3.14 PO Exclusion and Ranking

```

LD M1 MOV K-1 D_EligCasualTA
LDI M1 MOV D_ScoreCasualTA D_EligCasualTA
LD M2 MOV K-1 D_EligCasualTB
LDI M2 MOV D_ScoreCasualTB D_EligCasualTB

```

```

LD M3 MOV K-1 D_EligCasualTC
LDI M3 MOV D_ScoreCasualTC D_EligCasualTC

MOV K1 D_MaxID
MOV D_EligCasualTA D_MaxScore

LD> D_EligCasualTB D_MaxScore
OUT M_Challenger
LD M_Challenger MOV D_EligCasualTB D_MaxScore
LD M_Challenger MOV K2 D_MaxID

LD> D_EligCasualTC D_MaxScore
OUT M_Challenger
LD M_Challenger MOV D_EligCasualTC D_MaxScore
LD M_Challenger MOV K3 D_MaxID

LD= D_MaxID K1 MOV D_ThetaTA D_RankedTankTemp
LD= D_MaxID K2 MOV D_ThetaTB D_RankedTankTemp
LD= D_MaxID K3 MOV D_ThetaTC D_RankedTankTemp

```

3.15 Height-to-Mass Conversion

```

SUB K2000 D_DistT5 D_HeightT5
DMUL D_HeightT5 K24353 D510
DDIV D510 K10000 D_MassT5
// T10 tanks: SUB K2200 ..., DMUL ... K40455, same pattern

```

3.16 T_f Thermal Blending

```

MOV D_ThetaH D330 MOV K0 D331
MOV D_ThetaS D332 MOV K0 D333
MOV D_Mp D334 MOV K0 D335

DMUL K75808 D330 D340
DMUL D334 K2000 D344
DMUL D344 D332 D350

DADD D340 D350 D360
DADD K75808 D344 D362

DDIV D360 D362 D370
MOV D370 D_Tf

LD>= D_Tf K550
OUT M_TfInterlock
LD M_TfInterlock
OUT M4_ThrottleOrDivert

```

3.17 P1 – T5 Block

```

LD<  D_ThetaS K400
OUT  M_P1Demand
LD   M_P1Demand
OUT  Y11

LD   M_TfInterlock
OUT  M4_ThrottleOrDivert

LD<  D_AlphaS K50
AND>= D_RankedTankTemp K360
OUT  M_BatchRelease
LD   M_BatchRelease
OUT  Y5

```

Y11 is driven twice: once here, directly from M_P1Demand, and again in the Need-Array Arbiter below from M_Serve_T5. Since M_Serve_T5 is itself derived from M_D_T5 = M_P1Demand, the two rungs agree in steady state, but the arbiter's assignment executes later in the scan and is the one that actually wins – making the OUT Y11 rung here redundant/dead. Recommend removing it from here (or from the arbiter, keeping whichever placement is intended as canonical) so there is a single point of truth for Y11.

3.18 P2 – PO Tank Block

```

LD  D_DispensedVol
AND>= D_DispensedVol D_2xNd
OUT M_Overrun_Pulse

LD  M1  OR M2  OR M3
AND [dispense active, not yet "process complete"]
OUT M_P2Demand

```

3.19 Need-Array Arbiter

```

LD  M_P1Demand          OUT M_D_T5
LD  M1  AND M_P2Demand   OUT M_D_TA_asPO
LD  M2  AND M_P2Demand   OUT M_D_TB_asPO
LD  M3  AND M_P2Demand   OUT M_D_TC_asPO

LDI M1  AND< D_ThetaTA K360  OUT M_D_TA_asMelt
LDI M2  AND< D_ThetaTB K360  OUT M_D_TB_asMelt
LDI M3  AND< D_ThetaTC K360  OUT M_D_TC_asMelt

LD M_D_TA_asPO  OR M_D_TA_asMelt  OUT M_D_TA
LD M_D_TB_asPO  OR M_D_TB_asMelt  OUT M_D_TB
LD M_D_TC_asPO  OR M_D_TC_asMelt  OUT M_D_TC

```

```
MOV K0 D_NeedCount
LD M_D_T5   INC D_NeedCount
LD M_D_TA   INC D_NeedCount
LD M_D_TB   INC D_NeedCount
LD M_D_TC   INC D_NeedCount

LD M_D_T5   OUT M_Serve_T5
LD M_D_TA   OUT M_Serve_TA
LD M_D_TB   OUT M_Serve_TB
LD M_D_TC   OUT M_Serve_TC

LD= D_NeedCount K4   ANI M0   AND M1
LD< D_ScoreImmTB D_ScoreImmTC   RST M_Serve_TB
LD= D_NeedCount K4   ANI M0   AND M1
LD>= D_ScoreImmTB D_ScoreImmTC   RST M_Serve_TC

LD= D_NeedCount K4   ANI M0   AND M2
LD< D_ScoreImmTA D_ScoreImmTC   RST M_Serve_TA
LD= D_NeedCount K4   ANI M0   AND M2
LD>= D_ScoreImmTA D_ScoreImmTC   RST M_Serve_TC

LD= D_NeedCount K4   ANI M0   AND M3
LD< D_ScoreImmTA D_ScoreImmTB   RST M_Serve_TA
LD= D_NeedCount K4   ANI M0   AND M3
LD>= D_ScoreImmTA D_ScoreImmTB   RST M_Serve_TB

LD= D_NeedCount K4   AND M0
AND< D_ScoreImmTA D_ScoreImmTB   AND< D_ScoreImmTA D_ScoreImmTC
RST M_Serve_TA
LD= D_NeedCount K4   AND M0
AND< D_ScoreImmTB D_ScoreImmTA   AND< D_ScoreImmTB D_ScoreImmTC
RST M_Serve_TB
LD= D_NeedCount K4   AND M0
AND< D_ScoreImmTC D_ScoreImmTA   AND< D_ScoreImmTC D_ScoreImmTB
RST M_Serve_TC

MOV K0 D_ServeCount
LD M_Serve_T5   INC D_ServeCount
LD M_Serve_TA   INC D_ServeCount
LD M_Serve_TB   INC D_ServeCount
LD M_Serve_TC   INC D_ServeCount

LD M2_SystemOK   AND M3_AutoEnabled   AND= D_ServeCount K1   OUT Y6
LD M2_SystemOK   AND M3_AutoEnabled   AND= D_ServeCount K2   OUT Y2
LD M2_SystemOK   AND M3_AutoEnabled   AND= D_ServeCount K3   OUT Y17

LD M_Serve_T5   OUT Y11   LD M_Serve_TA   OUT Y12
LD M_Serve_TB   OUT Y13   LD M_Serve_TC   OUT Y14
```

```
LD M2_SystemOK
ANI M3_AutoEnabled
AND X2
OUT Y2
```

3.20 Solar and Auxiliary Gating

```
LD M3_AutoEnabled
AND> D_ThetaSol D_Twater_target
OUT Y15
```

```
LDI M3_AutoEnabled
AND X10
OUT Y15
```

```
LD M3_AutoEnabled
AND<= D_ThetaSol D_Twater_target
OUT Y16
```

```
LDI M3_AutoEnabled
ANI X10
OUT Y16
```

```
LD Y15
OUT Y25
```

3.21 Oil Distribution

```
LD M_DemandX
OUT M_ServeX
```

```
LDI M_DemandX AND M_DemandY
OUT M_ServeY
```

```
LDI M_DemandX ANI M_DemandY AND M_DemandZ
OUT M_ServeZ
```

```
LD M_ServeX OUT Y22
LD M_ServeY OUT Y21
LD M_ServeY OUT Y24
LD M_ServeZ OUT Y21
LD M_ServeZ OUT Y23
```

```
LD M_ServeX OR M_ServeY OR M_ServeZ
OUT Y3
```

Per the Tag ID table, Y21–Y24 = PBV100–103 = T5/TA/TB/TC. As written, ServeX opens only Y22 (TA) with no T5 outlet valve (Y21); ServeY and ServeZ both open Y21 in addition to their own tank valve. If X/Y/Z map onto TA/TB/TC in that order, confirm whether ServeX is genuinely meant to bypass the T5 outlet valve, or whether a rung driving Y21 from M_ServeX is missing.

3.22 Fault Detection

```
// --- TANK A FAULT DETECTION ---
LD  M10_Refresh
SUB D_ThetaTA D_ThetaTA_Prev D_DeltaThetaTA
LD  M10_Refresh
MOV D_ThetaTA D_ThetaTA_Prev

DMUL K5011  K100  D600
DDIV D600  D_MassTA D602
DDIV D602  K2000  D_MaxHeatRateTA

SUB  D_ThetaTA K181 D610
DMUL K668  D610  D620
DDIV D620  D_MassTA D622
DDIV D622  K2000  D_MaxCoolRateTA

LD>  D_DeltaThetaTA D_MaxHeatRateTA
OUT  M_FaultHeatingTA
LD<  D_DeltaThetaTA KO
AND> D_DeltaThetaTA D_MaxCoolRateTA
OUT  M_FaultCoolingTA

LD  M10_Refresh
SUB D_MassTA D_MassTA_Prev D_DeltaMassTA
LD  M10_Refresh
MOV D_MassTA D_MassTA_Prev

LD  M_ServeTA_AODD
OR  M_ServeX  OR M_ServeY  OR M_ServeZ
OUT M_TAActiveTransfer

LDI M_TAActiveTransfer
AND<> D_DeltaMassTA KO
OUT M_FaultLevelTA

LD>  D_ThetaTA K750
OUT  M_HighLimitTA

// --- TANK B FAULT DETECTION ---
LD  M10_Refresh
SUB D_ThetaTB D_ThetaTB_Prev D_DeltaThetaTB
```

```
LD M10_Refresh
MOV D_ThetaTB D_ThetaTB_Prev

DMUL K5011 K100 D630
DDIV D630 D_MassTB D632
DDIV D632 K2000 D_MaxHeatRateTB

SUB D_ThetaTB K181 D640
DMUL K668 D640 D650
DDIV D650 D_MassTB D652
DDIV D652 K2000 D_MaxCoolRateTB

LD> D_DeltaThetaTB D_MaxHeatRateTB
OUT M_FaultHeatingTB
LD< D_DeltaThetaTB K0
AND> D_DeltaThetaTB D_MaxCoolRateTB
OUT M_FaultCoolingTB

LD M10_Refresh
SUB D_MassTB D_MassTB_Prev D_DeltaMassTB
LD M10_Refresh
MOV D_MassTB D_MassTB_Prev

LD M_ServeTB_A0DD
OR M_ServeX OR M_ServeY OR M_ServeZ
OUT M_TBActiveTransfer

LDI M_TBActiveTransfer
AND<> D_DeltaMassTB K0
OUT M_FaultLevelTB

LD> D_ThetaTB K750
OUT M_HighLimitTB

// --- TANK C FAULT DETECTION ---
LD M10_Refresh
SUB D_ThetaTC D_ThetaTC_Prev D_DeltaThetaTC
LD M10_Refresh
MOV D_ThetaTC D_ThetaTC_Prev

DMUL K5011 K100 D660
DDIV D660 D_MassTC D662
DDIV D662 K2000 D_MaxHeatRateTC

SUB D_ThetaTC K181 D670
DMUL K668 D670 D680
DDIV D680 D_MassTC D682
DDIV D682 K2000 D_MaxCoolRateTC
```



```
LD> D_DeltaThetaTC D_MaxHeatRateTC
OUT M_FaultHeatingTC
LD< D_DeltaThetaTC K0
AND> D_DeltaThetaTC D_MaxCoolRateTC
OUT M_FaultCoolingTC

LD M10_Refresh
SUB D_MassTC D_MassTC_Prev D_DeltaMassTC
LD M10_Refresh
MOV D_MassTC D_MassTC_Prev

LD M_ServeTC_AODD
OR M_ServeX OR M_ServeY OR M_ServeZ
OUT M_TCActiveTransfer

LDI M_TCActiveTransfer
AND<> D_DeltaMassTC K0
OUT M_FaultLevelTC

LD> D_ThetaTC K750
OUT M_HighLimitTC

// --- MASTER ALARM AND HEATER SHUTOFF ---
LD M_HighLimitTA
OR M_HighLimitTB
OR M_HighLimitTC
SET M_Alarm

LD M_Alarm
RST Y7
RST Y10
RST Y20
```

3.23 Alarm Acknowledge and Reset

```
// Wait for operator reset input (Push-button or HMI)
LD X_AlarmReset

// Ensure all tank temperatures have dropped below the 75C limit
ANI M_HighLimitTA
ANI M_HighLimitTB
ANI M_HighLimitTC

// Unlatch the Master Alarm
RST M_Alarm

// Note: Once M_Alarm is reset, the interlocks on Y7, Y10, and Y20
// are removed, allowing the auxiliary stage register (Section 1.9)
```

```
// to automatically resume normal heater control if required.
```

4 Structured Text (ST) – EXPERIMENTAL

This section is experimental and unverified. Two separate caveats apply: (1) the author has no prior ST experience – this translation has not been sanity-checked against working PLC knowledge, only against the ladder logic it is derived from; (2) **GX Works2’s ST support is primarily associated with Mitsubishi’s iQ-R/iQ-F hardware family, not the classic FX-series line the ladder logic in this document assumes.** Whether the Coolmay L01S (already an unconfirmed FX3U-compatibility assumption for ladder) supports ST at all, and via which toolchain, has not been established. **Do not treat this section as ready to compile or run** – verify both the language availability and the syntax against the actual target environment before using it for anything beyond a logic cross-check. **Coverage is also partial by design:** First-Scan Initialization, AI Channel Read/Scale, the Manual/Automatic Gating Pattern, the 10-Second Refresh Oscillator, P2 – PO Tank Block, and Alarm Acknowledge and Reset are omitted here as straightforward, low-value translations; every block with any nontrivial arithmetic or branching is included.

4.1 Master Safety and Mode Control

```
IF NOT X0 THEN
    M1_ESTOP := TRUE;
END_IF;
IF X_EStopReset THEN
    M1_ESTOP := FALSE;
END_IF;
Y0 := M1_ESTOP;
M2_SystemOK := NOT M1_ESTOP;

Y1 := X1;
M3_AutoEnabled := X1;
```

4.2 PO Tank Designation

```
(* M100 = rising-edge pulse of X_Confirm; verify the correct
   edge-detection idiom for the target ST environment before use *)
IF M100 THEN
    D51 := D50;
END_IF;

M0 := (D51 = 0);
M1 := (D51 = 1);
M2 := (D51 = 2);
M3 := (D51 = 3);
```

4.3 T_{water} Target

```

IF M10_Refresh THEN
    D204 := 400 + (D_Ptotal * 300) / 6160;
    IF D204 < 400 THEN
        D204 := 400;
    END_IF;
    IF D204 > 700 THEN
        D204 := 700;
    END_IF;
    D_Twater_target := D204;
END_IF;

```

4.4 Mode Register

```

CASE D_Mode OF
    0: IF D_AlphaS < 85 THEN D_Mode := 1; END_IF;
    1: IF D_AlphaS >= 90 THEN D_Mode := 0;
        ELSIF D_AlphaS < 65 THEN D_Mode := 2;
        END_IF;
    2: IF D_AlphaS >= 70 THEN D_Mode := 1;
        ELSIF D_AlphaS < 45 THEN D_Mode := 3;
        END_IF;
    3: IF D_AlphaS >= 50 THEN D_Mode := 2; END_IF;
END_CASE;

```

4.5 Suspicion Score

```

IF M_Overrun_Pulse THEN
    D_Suspicion := D_Suspicion + 1;
END_IF;
IF D_Suspicion > 3 THEN
    D_Suspicion := 3;
END_IF;

M_HMIPrompt    := (D_Suspicion = 1);
M_Alarm         := (D_Suspicion = 2) OR (D_Suspicion = 3);
M_DisableFeed  := (D_Suspicion = 3);

(* T5, 5-min no-response auto-increment -- verify the correct timer
   function block (e.g. TON) and instantiation syntax for the target
   environment; represented conceptually only *)
IF M_PromptActive AND T5.Q THEN
    D_Suspicion := D_Suspicion + 1;
    M_PromptActive := FALSE;
END_IF;

IF M_DailyReset_Pulse THEN
    D_Suspicion := 0;

```

```
END_IF;
```

4.6 Dynamic Power Evaluation

```
IF M10_Refresh THEN
    D_Pbase := D_Etotal / 10;

    IF D_Twater < 400 THEN
        D_Ereq_kW := D_Pbase;
    ELSIF D_Twater >= 700 THEN
        D_Ereq_kW := 0;
    ELSE
        D_TempDiff := 700 - D_Twater;
        D_Ereq_kW := (D_Pbase * D_TempDiff) / 300;
    END_IF;
END_IF;
```

4.7 Auxiliary Stage Register

```
CASE D_AuxStage OF
    0: IF D_Ereq_kW > 0 THEN D_AuxStage := 1; END_IF;
    1: IF D_Ereq_kW <= 2 THEN D_AuxStage := 0;
        ELSIF D_Ereq_kW >= 14 THEN D_AuxStage := 2;
        END_IF;
    2: IF D_Ereq_kW < 10 THEN D_AuxStage := 1;
        ELSIF D_Ereq_kW >= 26 THEN D_AuxStage := 3;
        END_IF;
    3: IF D_Ereq_kW < 22 THEN D_AuxStage := 2; END_IF;
END_CASE;

Y7 := (D_AuxStage >= 1);
Y10 := (D_AuxStage >= 2);
Y20 := (D_AuxStage >= 3);
```

4.8 Tank Scoring and Ranking (worked for all three tanks)

```
D_TnormTA := (D_ThetaTA * 100) / 700;
D_TnormTB := (D_ThetaTB * 100) / 700;
D_TnormTC := (D_ThetaTC * 100) / 700;

D_ScoreCasualTA := (D_AlphaTA * 6) + (D_TnormTA * 4);
D_ScoreCasualTB := (D_AlphaTB * 6) + (D_TnormTB * 4);
D_ScoreCasualTC := (D_AlphaTC * 6) + (D_TnormTC * 4);

D_ScoreImmTA := (D_AlphaTA * 4) + (D_TnormTA * 6);
D_ScoreImmTB := (D_AlphaTB * 4) + (D_TnormTB * 6);
D_ScoreImmTC := (D_AlphaTC * 4) + (D_TnormTC * 6);

D_EligCasualTA := SEL(M1, D_ScoreCasualTA, -1);
```

```

D_EligCasualTB := SEL(M2, D_ScoreCasualTB, -1);
D_EligCasualTC := SEL(M3, D_ScoreCasualTC, -1);
(* SEL(condition, value_if_false, value_if_true) -- verify exact
   argument order for the target ST environment; IF/THEN/ELSE is the
   safer, more portable substitute if SEL's semantics are uncertain *)

D_MaxID := 1;
D_MaxScore := D_EligCasualTA;
IF D_EligCasualTB > D_MaxScore THEN
    D_MaxScore := D_EligCasualTB;
    D_MaxID := 2;
END_IF;
IF D_EligCasualTC > D_MaxScore THEN
    D_MaxScore := D_EligCasualTC;
    D_MaxID := 3;
END_IF;

CASE D_MaxID OF
    1: D_RankedTankTemp := D_ThetaTA;
    2: D_RankedTankTemp := D_ThetaTB;
    3: D_RankedTankTemp := D_ThetaTC;
END_CASE;

```

Note on ST vs. ladder for this block specifically. The ranking logic above is meaningfully cleaner in ST than in ladder – straightforward IF/THEN comparison replaces the flag-mediated “safe pattern” (Section 3.13 of the architecture document) needed in ladder to avoid the compare-after- overwrite bug. This is a genuine advantage of ST for this specific kind of logic, not just a stylistic difference.

4.9 Height-to-Mass Conversion

```

D_HeightT5 := 2000 - D_DistT5;
D_MassT5   := (D_HeightT5 * 24353) / 10000;

D_HeightTA := 2200 - D_DistTA;
D_MassTA   := (D_HeightTA * 40455) / 10000;
(* repeat identically for TB, TC *)

```

4.10 T_f Thermal Blending

```

(* 32-bit overflow risk noted in the ladder version applies identically
   here -- confirm the target ST environment's default integer width
   (many IEC platforms default to 32-bit DINT, which may remove the
   explicit zero-extension step the ladder version required) *)
D_Tf := (75808 * D_ThetaH + D_Mp * 2000 * D_ThetaS) / (75808 + D_Mp * 2000);

M_TfInterlock := (D_Tf >= 550);
M4_ThrottleOrDivert := M_TfInterlock;

```

4.11 P1 – T5 Block

```
M_P1Demand := (D_ThetaS < 400);
(* Y11 is driven from M_Serve_T5 in the Need-Array Arbiter below, not
   here -- see the corresponding note on the ladder version
   (Section~\ref{sec:safety-ladder}'s sibling P1 block) for why this
   direct assignment is intentionally omitted in the ST version. *)
```

```
M4_ThrottleOrDivert := M_TfInterlock;
```

```
M_BatchRelease := (D_AlphaS < 50) AND (D_RankedTankTemp >= 360);
Y5 := M_BatchRelease;
```

4.12 Need-Array Arbiter

```
M_D_T5 := M_P1Demand;
M_D_TA := (M1 AND M_P2Demand) OR (NOT M1 AND (D_ThetaTA < 360));
M_D_TB := (M2 AND M_P2Demand) OR (NOT M2 AND (D_ThetaTB < 360));
M_D_TC := (M3 AND M_P2Demand) OR (NOT M3 AND (D_ThetaTC < 360));
```

```
D_NeedCount := 0;
IF M_D_T5 THEN D_NeedCount := D_NeedCount + 1; END_IF;
IF M_D_TA THEN D_NeedCount := D_NeedCount + 1; END_IF;
IF M_D_TB THEN D_NeedCount := D_NeedCount + 1; END_IF;
IF M_D_TC THEN D_NeedCount := D_NeedCount + 1; END_IF;
```

```
M_Serve_T5 := M_D_T5;
M_Serve_TA := M_D_TA;
M_Serve_TB := M_D_TB;
M_Serve_TC := M_D_TC;
```

```
IF (D_NeedCount = 4) THEN
  IF M1 THEN
    IF D_ScoreImmTB < D_ScoreImmTC THEN M_Serve_TB := FALSE;
    ELSE M_Serve_TC := FALSE;
    END_IF;
  ELSIF M2 THEN
    IF D_ScoreImmTA < D_ScoreImmTC THEN M_Serve_TA := FALSE;
    ELSE M_Serve_TC := FALSE;
    END_IF;
  ELSIF M3 THEN
    IF D_ScoreImmTA < D_ScoreImmTB THEN M_Serve_TA := FALSE;
    ELSE M_Serve_TB := FALSE;
    END_IF;
  ELSIF M0 THEN
    IF (D_ScoreImmTA <= D_ScoreImmTB) AND (D_ScoreImmTA <= D_ScoreImmTC) THEN
      M_Serve_TA := FALSE;
    ELSIF (D_ScoreImmTB <= D_ScoreImmTA) AND (D_ScoreImmTB <= D_ScoreImmTC) THEN
      M_Serve_TB := FALSE;
```

```

        ELSE
            M_Serve_TC := FALSE;
        END_IF;
    END_IF;
END_IF;

D_ServeCount := 0;
IF M_Serve_T5 THEN D_ServeCount := D_ServeCount + 1; END_IF;
IF M_Serve_TA THEN D_ServeCount := D_ServeCount + 1; END_IF;
IF M_Serve_TB THEN D_ServeCount := D_ServeCount + 1; END_IF;
IF M_Serve_TC THEN D_ServeCount := D_ServeCount + 1; END_IF;

IF M2_SystemOK AND M3_AutoEnabled THEN
    Y6 := (D_ServeCount = 1);
    Y2 := (D_ServeCount = 2);
    Y17 := (D_ServeCount = 3);
END_IF;

Y11 := M_Serve_T5;
Y12 := M_Serve_TA;
Y13 := M_Serve_TB;
Y14 := M_Serve_TC;

```

The four-way tie-break is the clearest example of ST's advantage over ladder in this whole document. The ladder version needed six separate rung groups with duplicated D_NeedCount=4 conditions; the ST version expresses the same logic as one readable IF/ELSIF chain matching the plain-English rule directly ("T5 guaranteed, PO tank guaranteed if it exists, drop the lower S_immediate of whichever remains"). If ST is confirmed available on the target hardware, this block in particular is worth prioritising for an ST implementation even if the rest of the program stays in ladder.

4.13 Solar and Auxiliary Gating

```

IF M3_AutoEnabled THEN
    Y15 := (D_ThetaSol > D_Twater_target);
    Y16 := (D_ThetaSol <= D_Twater_target);
ELSE
    Y15 := X10;
    Y16 := NOT X10;
END_IF;
Y25 := Y15;

```

4.14 Oil Distribution

```

M_ServeX := M_DemandX;
M_ServeY := (NOT M_DemandX) AND M_DemandY;
M_ServeZ := (NOT M_DemandX) AND (NOT M_DemandY) AND M_DemandZ;

```

```

Y22 := M_ServeX;
Y21 := M_ServeY OR M_ServeZ;
Y24 := M_ServeY;
Y23 := M_ServeZ;
Y3  := M_ServeX OR M_ServeY OR M_ServeZ;

```

4.15 Fault Detection

```

IF M10_Refresh THEN
    D_DeltaThetaTA := D_ThetaTA - D_ThetaTA_Prev;
    D_ThetaTA_Prev := D_ThetaTA;

    D_MaxHeatRateTA := (5011 * 100) / D_MassTA / 2000;
    D_MaxCoolRateTA := (668 * (D_ThetaTA - 181)) / D_MassTA / 2000;

    M_FaultHeatingTA := (D_DeltaThetaTA > D_MaxHeatRateTA);
    M_FaultCoolingTA := (D_DeltaThetaTA < 0) AND
        (ABS(D_DeltaThetaTA) > D_MaxCoolRateTA);

    D_DeltaMassTA := D_MassTA - D_MassTA_Prev;
    D_MassTA_Prev := D_MassTA;
END_IF;

(* repeat identically for TB, TC -- same as the Height-to-Mass block,
   only D_DeltaThetaTA/D_MaxHeatRateTA/D_MaxCoolRateTA/
   M_FaultHeatingTA/M_FaultCoolingTA/D_DeltaMassTA and their _Prev
   registers are shown; TB and TC follow the identical pattern using
   the ladder version's D630--D682 constants for TB and D660--D682-style
   constants for TC (Section~3.21) *)

M_TAACTiveTransfer := M_ServeTA_AODD OR M_ServeX OR M_ServeY OR M_ServeZ;
M_FaultLevelTA := (NOT M_TAACTiveTransfer) AND (D_DeltaMassTA <> 0);
(* M_TBActiveTransfer/M_FaultLevelTB and M_TCACTiveTransfer/
   M_FaultLevelTC follow the same pattern, substituting each tank's own
   AODD-serve flag and delta-mass register *)

M_HighLimitTA := (D_ThetaTA > 750);
M_HighLimitTB := (D_ThetaTB > 750);
M_HighLimitTC := (D_ThetaTC > 750);

IF M_HighLimitTA OR M_HighLimitTB OR M_HighLimitTC THEN
    M_Alarm := TRUE;
    Y7  := FALSE;
    Y10 := FALSE;
    Y20 := FALSE;
END_IF;

```


5 Conclusion

1. **Every ladder block in the architecture document is represented here in full**, organised for lookup rather than narrative reading – use the ID tables in Section 2 to locate any tag, then the matching subsection in Section 3 for its logic.
2. **The ST section is a genuine starting point, not a finished artefact, and its coverage is intentionally partial** (see the note at the top of Section 4). Confirm ST availability and exact syntax on the actual target hardware/toolchain before relying on it, and treat the two flagged resultboxes (tank ranking, four-way tie-break) as the strongest early candidates if a partial ST/ladder hybrid is ever considered.
3. **Two open items surfaced directly by this pass, still unresolved:** Y11 is written from two different places in the ladder listing with the arbiter's assignment silently overriding the P1 block's; and the oil-distribution valve mapping leaves ServeX without a T5 outlet-valve rung while ServeY/ServeZ both include one. Each is flagged in place in Section 3 above and should be confirmed with the process design before this goes to hardware. (The jacket pump, Y4/X4, is now resolved: it runs continuously off M2_SystemOK – see Section 3.5.)
4. **Every open item from the architecture document still applies here unchanged** – this document consolidates *how* to implement what has been decided; it does not resolve anything that was still open when it was written.

Prepared by **Sambridda Ranabhat** · Mechatronics Engineering Pvt. Ltd. (MEPL),
Kathmandu, Nepal

Based on the *VCH Sizing Framework, Second Edition*
DOI: 10.5281/zenodo.21009246